

WebSembly Instruction Set Architecture

Instruction	Description
LDR Rd, <memory ref>	Load the value stored in the memory location specified by <memory ref> into register d.
STR Rd, <memory ref>	Store the value that is in register d into the memory location specified by <memory ref>.
ADD Rd, Rn, <operand 2>	Add the value specified in <operand 2> to the value in register n and store the result in register d.
SUB Rd, Rn, <operand 2>	Subtract the value specified by <operand 2> from the value in register n and store the result in register d.
MOV Rd, <operand 2>	Copy the value specified by <operand 2> into register d.
CMP Rn, <operand 2>	Compare the value stored in register n with the value specified by <operand 2>.
B <label>	Always branch to the instruction at position <label> in the program.
B<condition> <label>	Branch to the instruction at position <label> if the last comparison met the criterion specified by <condition>. Possible values for <condition> and their meanings are: EQ: equal to NE: not equal to GT: greater than LT: less than
AND Rd, Rn, <operand 2>	Perform a bitwise logical AND operation between the value in register n and the value specified by <operand 2> and store the result in register d.
ORR Rd, Rn, <operand 2>	Perform a bitwise logical OR operation between the value in register n and the value specified by <operand 2> and store the result in register d.

Instruction	Description
XOR Rd, Rn, <operand 2>	Perform a bitwise logical XOR (exclusive or) operation between the value in register n and the value specified by <operand 2> and store the result in register d.
NOT Rd, <operand 2>	Perform a bitwise logical NOT operation on the value specified by <operand 2> and store the result in register d.
LSL Rd, Rn, <operand 2>	Logically shift left the value stored in register n by the numbers of bits specified by <operand 2> and store the result in register d.
LSR Rd, Rn, <operand 2>	Logically shift right the value stored in register n by the numbers of bits specified by <operand 2> and store the result in register d.
HALT	Stops the execution of the program.

Labels

A label is placed in the code by writing an identifier followed by a colon (:). To refer to a label the identifier of the label is placed after the branch instruction.

Operand 2

<operand 2> can be interpreted in two different ways, depending on whether the first character is a # or an R:

- # – use the value specified after the #, e.g #25, means use the value 25
- Rm – use the value stored in register m, e.g R6, means use the value in register 6

There are 16 general purpose registers available for use, numbered 0 to 16. Additionally, <memory ref> can also be specified with the use of an immediate (#) or register (R) address.